CSS452 Practical Midterm: Cheatsheet

Open-book reference for the practical exam. Everything here comes from Lecture Notes 3–6, Exercises 1–4 and Homework 1–4. Where a line is **not** from the course material it is marked *(not in the lecture)*.

Every code listing below also exists as a ready-to-open sketch in `Midterm/sketches/<name>/<name>.ino`. In the exam, open the sketch, change the pins, upload. Do not retype.

---

0. The exam in one screen

What	Detail
Length / weight	1 hour, 42 points, 10% of the course
Questions	3
Covers	Lecture Notes 3–6, Quizzes 3–6, Exercises 1–4, Homework 1–4
Format	Open book. You connect circuits **and** write Arduino code
Bring	Notebook computer, your equipment box (no spares are provided)
Hand in	Paper work → the instructor. Code **and** recorded videos → Google Classroom
Penalties	Late arrival and late submission both lose points. Group 2 arriving late is not allowed in and scores 0

Source: `CSS452 – Information of the Midterm Examination.pdf`, section 2.

Not covered by this sheet: Quizzes 3–6. They are in the exam scope but were never dropped into this repo, so nothing here is drawn from them.

Before you walk in

1. **Do one full Verify + Upload on the laptop you will bring.** Not last month's — today's. An OS update can break the toolchain without telling you.
2. `PubSubClient` library installed (Sketch → Include Library → Manage Libraries). Without it no MQTT sketch compiles. (LN6 slide 15)
3. Board: Tools → Board → **ThaiEasyElec's ESPino32**. Serial Monitor at **115200**. (LN3 slides 18, 21)
4. Phone hotspot on **2.4 GHz**, screen kept awake, name and password known by heart. The ESP32 cannot join 5 GHz.
5. ThingSpeak channel already created with **Field 1 and Field 2 ticked**, Write API Key copied somewhere you can paste from. (LN5 slides 94–95)
6. Your student ID ready to paste into MQTT `ClientID` and `Topic`.
7. Equipment box: ESP32, USB cable, breadboard, LEDs, 330 Ω, 10 kΩ, push switches, potentiometer, jumper wires, and a second ESP32 if you own one.
8. This folder available **offline** (`CHEATSHEET.html` opens without internet).

Sixty minutes, three questions

About 20 minutes each, including wiring, code, video and upload to Classroom. *(My suggestion, not from the guideline.)* Record the video the moment a question works — do not leave three videos for the last five minutes. If a question stalls for more than a few minutes, move on and come back.

Each video in the homework showed the same three things; expect the same: circuit connection, Arduino code, demonstration.

1. Find it fast

The question mentions	Open this sketch	Section
switch turns LED on/off	`LN3_Ex1_Digital_InOut`	3.1
read potentiometer, show on serial monitor	`LN3_Ex2_Analog_In`	3.2
LED brightness, PWM, dimming	`LN3_Ex3_Analog_PWM`	3.3
touch a wire / touch pin	`LN3_Ex4_Touch_Pin`, `MIX_Touch_Controls_LED`	3.4
hall sensor, magnet	`LN3_Ex5_Hall_Effect`	3.5
restart / reset the ESP32	`LN3_Soft_Reset`, `HW1_P3_WiFi_Touch_Restart`	3.6
connect to WiFi, show IP address	`LN3_Ex6_WiFi_Connection`	3.7
ESP32 as access point, count stations	`LN3_Ex7_WiFi_SoftAccessPoint`	3.8
date and time, NTP	`LN3_Ex8_DateTimerNTP`	3.9
blink without `delay()`, `millis()`	`LN4_Ex1_millis`	4.1
interrupt, toggle on press	`LN4_Ex2_Interrupts`	4.2
tasks, `xTaskCreate`	`LN4_Ex3_Multitasking`	4.3
which core is running	`LN4_Ex4_ShowCore`	4.4
assign a task to core 0 / core 1	`LN4_Ex5_AssignCore`	4.5
deep sleep	`LN4_Ex6_DeepSleep`	4.6
wake up after N seconds	`LN4_Ex7_TimerWakeUp`	4.7
wake up by touching	`LN4_Ex8_TouchWakeUp`	4.8
wake up by one switch	`LN4_Ex9_ext0WakeUp`	4.9
wake up by several switches	`LN4_Ex10_ext1WakeUp`	4.10
button puts it to sleep, timer wakes it	`HW2_P3_Tasks_SleepButton_TimerWake`	4.11
web page with buttons controlling LEDs	`LN5_S3_WebServer_ControlOutputs`, `HW3_P3_WebServer_ThreeLEDs`	5.1
web page showing a sensor value	`LN5_S4_WebServer_MonitorSensor`	5.2
web page with buttons **and** a sensor	`MIX_WebServer_LEDs_and_Sensor`	5.4
ThingSpeak, cloud, charts	`LN5_S5_WebClient_ThingSpeak`	5.5
MQTT publisher	`LN6_S3_PubNode`	6.1
MQTT subscriber	`LN6_S3_SubNode`	6.2

The question mentions	Open this sketch	Section
MQTT publish and subscribe on one board	`LN6_S4_PubSub`	6.3
two boards controlling each other	`HW4_Q2_Node1`, `HW4_Q2_Node2`	6.4

2. Board, pins and circuits

2.1 The board

ThaiEasyElec's **ESPino32** (ESP-WROOM-32). Dual-core 240 MHz, 2.4 GHz WiFi and Bluetooth. There is an **on-board LED on pin 16**, a RESET button and a PROGRAM button. (LN3 slides 5–6)

Upload stuck at `Connecting…` → press the **PROGRAM** button on the board. (LN3 slide 20)

2.2 Pin labels → GPIO numbers

The board prints labels; the code needs GPIO numbers (or the `A0` name for analog). (LN3 slides 8–9)

Header P1 (label)	GPIO		Header P2 (label)	GPIO
GND	—		3V3	—
3V3	—		GND	—
EN	enable		SCL / 22	22
A0 / 36	36		SDA / 21	21
A3 / 39	39		Tx	3
A4 / T9 / **32**	32		Rx	1
A5 / T8 / 33	33		**17**	17
A6 / **34**	34		**16**	16
A7 / 35	35		SCK / 18	18
A10 / T0 / 4	4		MISO / 19	19

Header P3 (label)	GPIO		Header P4 (label)	GPIO
A11 / T1 / 0	0		MOSI / 23	23
A12 / T2 / **2**	2		SS / 5	5
A13 / T3 / 15	15		SD2 / 9	9
A14 / T4 / 13	13		SD3 / 10	10
A15 / T5 / 12	12		CMD / 11	11
A16 / T6 / 14	14		CLK / 6	6
A17 / T7 / **27**	27		SD0 / 7	7
A18 / DAC1 / 25	25		SD1 / 8	8
A19 / DAC2 / **26**	26		GND	—
Vin	—		VBAT	—

Pins the course actually used: **16, 17** (LEDs), **26, 27** (LEDs / SW1), **32** (touch wire, wake switch, third LED), **34** (switch), **2** (ext1 switch), **A0** (potentiometer).

2.3 What each pin can do

Capability	Pins	Source
Analog input	`A0` – `A19` ; 0–3.3 V reads as 0–4095	LN3 slide 28
PWM output	GPIO 0–19, 21–23, 25–27, 32–33; value 0–255	LN3 slide 32
Touch	GPIO 0, 2, 4, 12, 13, 14, 15, 27, 32, 33	LN3 slide 37
Pin-change interrupt	every GPIO **except 6–11**	LN4 slide 14
ext0 / ext1 wake-up	GPIO 0, 2, 4, 12–15, 25–27, 32–39	LN4 slides 56, 61
Hall sensor	built in, no pin	LN3 slide 41

Three traps *(general ESP32 facts, not in the lecture)*:

- **GPIO 34, 35, 36, 39 are input-only.** They cannot drive an LED. Slide 7 calls these "I pins → only input".
- **Analog pins A10–A19 stop working while WiFi is on** (they sit on ADC2, which WiFi takes over). With any WiFi, web or MQTT sketch, read analog on **A0, A3, A4, A5, A6 or A7**. Every course example uses `A0`.
- **GPIO 6–11 are wired to the flash chip.** Leave them alone.

2.4 Circuits

All from the lecture schematics. Resistor values are the lecture's: **10 kΩ** with a switch, **330 Ω** with an LED.

LED — long leg to the GPIO, short leg through 330 Ω to GND. `HIGH` lights it.

```
GPIO ------>|------[330]------ GND
       long   short
       leg    leg
```

Switch, pressing gives LOW — 10 kΩ pull-up to 3v3, switch to GND. Used with GPIO 34 (LN3 Ex 1, LN4 Ex 1–5), GPIO 32 (LN4 Ex 9), GPIO 26 (HW2 P3).

```
3v3 ---[10k]---+-------- GPIO       not pressed -> HIGH
               |                    pressed     -> LOW
              [SW]
               |
              GND
```

Potentiometer — outer legs to 3v3 and GND, centre leg (wiper) to `A0`.

```
3v3 ------ outer leg  \
                       [POT]  centre leg (wiper) ------ A0
GND ------ outer leg  /
```

Touch wire — one bare jumper wire in GPIO 32. Nothing else. (LN3 slide 39)

Two switches for ext1, pressing gives HIGH — LN4 slide 64, GPIO 2 and 32.

```
3v3 ---[10k]---[SW]---+-------- GPIO 2     (and the same again on GPIO 32)
                      |
                    [330]
                      |
                     GND
```

> **Check this one on the bench.** The slide labels the top resistor 10K and the bottom one 330, and says pressing gives HIGH. With those values a pressed switch puts only about 0.1 V on the pin (3.3 V × 330 / 10 330), which reads LOW. A pull-down normally has the *large* resistor at the bottom. If `ESP_EXT1_WAKEUP_ANY_HIGH` never wakes the board, that is the first thing to suspect; ask the instructor rather than guess on the paper.

The standard breadboards

Topic	Wiring	Source
LN3 Ex 1	SW → 34, LED → 17	LN3 slide 26
LN3 Ex 2	pot → A0	LN3 slide 30
LN3 Ex 3	pot → A0, LED → 17	LN3 slide 35
LN3 Ex 4, LN4 Ex 8	wire → 32	LN3 slide 39, LN4 slide 53
LN4 Ex 1–5	SW → 34, LED1 → 16, LED2 → 17	LN4 slides 4, 9, 15, 24
LN4 Ex 9	SW → 32	LN4 slide 57
LN4 Ex 10	SW1 → 2, SW2 → 32 (press = HIGH)	LN4 slide 64
HW2 P3	LN4 Ex 5 board + SW1 → 26	Homework 2
LN5 §3	LED1 → 26, LED2 → 27	LN5 slide 52
LN5 §4, §5.2	pot → A0	LN5 slides 76, 96
HW3 P3	LED1 → 16, LED2 → 17, LED3 → 32	Homework 3
LN6 §3	PubNode: pot → A0. SubNode: LED → 26	LN6 slide 20
LN6 §4, HW4 Q2	pot → A0 and LED → 26 on the same board	LN6 slide 41

3. Lecture Note 3: basics and WiFi

LN3 command reference

Do this	Command	Notes
Serial	`Serial.begin(115200);`	**115200, not 9600** (slide 19)
Pin as output	`pinMode(pin, OUTPUT);`	in `setup()`
Pin as input	`pinMode(pin, INPUT);`	in `setup()`
Write digital	`digitalWrite(pin, HIGH);`	HIGH ≈ 3.3 V, LOW ≈ 0 V
Read digital	`X = digitalRead(pin);`	1 or 0
Read analog	`X = analogRead(A0);`	0–4095, no `pinMode` needed
PWM set-up	`ledcSetup(ledChannel, freq, resolution);`	channel 0–15, freq 5000, resolution 8
PWM attach	`ledcAttachPin(PWMpin, ledChannel);`	in `setup()`
PWM write	`ledcWrite(ledChannel, dutycycle);`	0–255
Scale a range	`map(value, 0, 4095, 0, 255);`	analog → PWM
Touch	`X = touchRead(pin);`	0–1023; **touched = low**, untouched = high
Hall	`X = hallRead();`	−500 … 500
Restart	`ESP.restart();`	
WiFi header	`#include <WiFi.h>`	
Station mode	`WiFi.mode(WIFI_STA);`	
Connect	`WiFi.begin("ssid", "password");`	
Status	`WiFi.status()`	`WL_CONNECTED` = 3
Own IP	`WiFi.localIP()`	
Be an access point	`WiFi.softAP("ssid", "password");`	password ≥ 8 characters
Stations joined	`WiFi.softAPgetStationNum()`	
NTP	`configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);`	Thailand: `7*3600`, `0`, `"pool.ntp.org"`
Read the clock	`getLocalTime(&timeinfo)`	fills a `struct tm`

`WiFi.status()` codes (slide 47): 0 idle · 1 SSID not found · 3 connected · 4 wrong password · 6 disconnected / not in station mode.

3.1 Example 1: digital in and out

Press the switch → LED on. Circuit: SW → 34 (press = LOW), LED → 17. (slides 26–27)

Sketch: `sketches/LN3_Ex1_Digital_InOut/LN3_Ex1_Digital_InOut.ino`

```
// set pin numbers
const int buttonPin = 34;  // the push button pin
const int ledPin =  17;    // the LED pin

// variable for storing the pushbutton status
int buttonState = 0;

void setup() {
  Serial.begin(115200);
  // initialize the pushbutton pin as an input
  pinMode(buttonPin, INPUT);
  // initialize the LED pin as an output
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // read the state of the pushbutton value
  buttonState = digitalRead(buttonPin);
  Serial.println(buttonState);
  // check if the pushbutton is pressed.
  // if it is, the buttonState is LOW
  if (buttonState == LOW) {
    // turn LED on
    digitalWrite(ledPin, HIGH);
  } else {
    // turn LED off
    digitalWrite(ledPin, LOW);
  }
}
```

3.2 Example 2: analog in

Print the potentiometer value every second. Circuit: pot → A0. (slides 30–31)

Sketch: `sketches/LN3_Ex2_Analog_In/LN3_Ex2_Analog_In.ino`

```
const int potPin = A0; //Potentiometer pin

// Variable for storing the potentiometer value
int Value = 0;

void setup() {
  Serial.begin(115200);
  delay(1000);
}

void loop() {
  // Reading potentiometer value
  Value = analogRead(potPin);
  Serial.println(Value);
  delay(1000);
}
```

3.3 Example 3: PWM brightness

Potentiometer sets LED brightness. Circuit: pot → A0, LED → 17. (slides 35–36)

Sketch: `sketches/LN3_Ex3_Analog_PWM/LN3_Ex3_Analog_PWM.ino`

```
const int potPin = A0; //Potentiometer pin
const int ledPin = 17;  // LED pin

// setting PWM properties
const int freq = 5000;
const int ledChannel = 0;
const int resolution = 8;

// Variable for storing the potentiometer value
int potValue = 0;
int ledValue = 0;

void setup() {
  // configure LED PWM functionalitites
  ledcSetup(ledChannel, freq, resolution);

  // attach the channel to the GPIO to be controlled
  ledcAttachPin(ledPin, ledChannel);
}

void loop() {
  // Reading potentiometer value
  potValue = analogRead(potPin);
  ledValue = map(potValue, 0, 4095, 0, 255);

  // changing the LED brightness with PWM
  ledcWrite(ledChannel, ledValue);
}
```

The three PWM steps: **declare** channel/freq/resolution → `ledcSetup` + `ledcAttachPin` in `setup()` → `ledcWrite` in `loop()`.

> If the compiler says `'ledcSetup' was not declared in this scope`, the laptop has ESP32 core 3.x, where these two calls were replaced. Use `ledcAttach(ledPin, freq, resolution);` in `setup()` and `ledcWrite(ledPin, ledValue);` in `loop()`. *(Not in the lecture — the lecture's code is for core 2.x.)*

3.4 Example 4: touch pin

Print the touch value every second. Circuit: one wire in GPIO 32. (slides 39–40)

Sketch: `sketches/LN3_Ex4_Touch_Pin/LN3_Ex4_Touch_Pin.ino`

```
void setup() {
  Serial.begin(115200);
  delay(1000); // give me time to bring up serial monitor
  Serial.println("ESP32 Touch Test");
}

void loop() {
  Serial.println(touchRead(32));  // Touch sensor from Pin 32
  delay(1000);
}
```

The slide's sample output: about **38–46 not touched**, **6–9 touched**. The lecture never states a threshold — read your own board's two bands on the serial monitor first and pick a number between them.

Touch as a switch *(written for this sheet; combines Example 4 and Example 1)*:

Sketch: `sketches/MIX_Touch_Controls_LED/MIX_Touch_Controls_LED.ino`

```
const int touchPin = 32;  // touch-sensor pin
const int ledPin = 17;    // the LED pin

// TODO: set from your own readings. The lecture gives no threshold;
// slide 40 only shows about 38–46 untouched and 6–9 touched.
const int Threshold = 20;

int Value = 0;

void setup() {
  Serial.begin(115200);
  delay(1000); // give me time to bring up serial monitor
  pinMode(ledPin, OUTPUT);
}

void loop() {
  Value = touchRead(touchPin);
  Serial.println(Value);
  if (Value < Threshold) {
    // touching the wire returns a low value
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }
  delay(100);
}
```

3.5 Example 5: hall sensor

No circuit. Bring a magnet near the metal cap. (slides 41–42)

Sketch: `sketches/LN3_Ex5_Hall_Effect/LN3_Ex5_Hall_Effect.ino`

```
void setup() {
  Serial.begin(115200);
  delay(1000); // give me time to bring up serial monitor
  Serial.println("ESP32 Hall Effect Test");
}

void loop() {
  Serial.println(hallRead());  // Read the hall effect
  delay(1000);
}
```

> `hallRead()` does not exist in ESP32 core 3.x. *(Not in the lecture.)*

3.6 Software reset

Restarts every 10 seconds. No circuit. (slide 43)

Sketch: `sketches/LN3_Soft_Reset/LN3_Soft_Reset.ino`

```
void setup() {
  Serial.begin(115200);
  Serial.println("Restarting in 10 seconds");
  delay(10000);
  ESP.restart();
}

void loop() {
}
```

3.7 Example 6: WiFi connection

Join an access point and print the IP address. (slide 48)

Sketch: `sketches/LN3_Ex6_WiFi_Connection/LN3_Ex6_WiFi_Connection.ino`

```
#include <WiFi.h>

void setup() {
  Serial.begin(115200);
  Serial.println();
  WiFi.mode(WIFI_STA);
  WiFi.begin("Your_SSID", "Your_Password");
  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.print("Connected, IP address: ");
  Serial.println(WiFi.localIP());
}

void loop() {
}
```

This `WiFi.mode` → `WiFi.begin` → `while (WiFi.status() != WL_CONNECTED)` block is the opening of **every** WiFi, web, ThingSpeak and MQTT sketch in the course.

3.8 Example 7: soft access point

The ESP32 becomes an access point; prints how many stations joined. (slide 52)

Sketch: `sketches/LN3_Ex7_WiFi_SoftAccessPoint/LN3_Ex7_WiFi_SoftAccessPoint.ino`

```
#include <WiFi.h>

void setup() {
  Serial.begin(115200);
  Serial.println();
  Serial.print("Setting soft-AP ... ");
  boolean result = WiFi.softAP("Your_SSID", "Your_Password");
  if(result == true) {
    Serial.println("Ready");
  } else {
    Serial.println("Failed!");
  }
}

void loop() {
  Serial.printf("Stations connected = %d\n", WiFi.softAPgetStationNum());
  delay(3000);
}
```

3.9 Example 8: date and time from NTP

Prints local time every second. Needs internet. (slides 54–56)

Sketch: `sketches/LN3_Ex8_DateTimerNTP/LN3_Ex8_DateTimerNTP.ino`

```
#include <WiFi.h>
#include "time.h"

const char* ssid       = "Your SSID";
const char* password   = "Your Password";

const char* ntpServer = "pool.ntp.org";
const long  gmtOffset_sec = 7*3600;
const int   daylightOffset_sec = 0;

void printLocalTime() {
  struct tm timeinfo;
  if(!getLocalTime(&timeinfo)){
    Serial.println("Failed to obtain time");
    return;
  }
  Serial.println(&timeinfo, "%A, %B %d %Y %H:%M:%S");
}

void setup() {
  Serial.begin(115200);

  //connect to WiFi
  Serial.printf("Connecting to %s ", ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
      delay(500);
      Serial.print(".");
  }
  Serial.println(" CONNECTED");

  //init and get the time
  configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
  printLocalTime();

  //disconnect WiFi as it's no longer needed
  WiFi.disconnect(true);
  WiFi.mode(WIFI_OFF);
}

void loop() {
  delay(1000);
  printLocalTime();
}
```

Format letters: `%A` weekday · `%B` month name · `%d` day · `%Y` year · `%H` hour · `%M` minute · `%S` second. The NTP server is contacted once; after that the internal clock counts (slide 60).

To keep a piece in a variable (slide 60):

```
char timeHour[3];
strftime(timeHour, 3, "%H", &timeinfo);
```

3.10 Homework 1 Problem 3

Connect to WiFi and show the IP, then restart when the wire is touched. Circuit: wire → 32. *(Composed for the homework — Examples 4 and 6 plus the reset command. The threshold `20` is not from the lecture.)*

Sketch:
`sketches/HW1_P3_WiFi_Touch_Restart/HW1_P3_WiFi_Touch_Restart.ino`

```
#include <WiFi.h>

const char* ssid     = "Your_SSID";
const char* password = "Your_Password";

const int touchPin = 32;  // touch-sensor pin, per Example 4 (slide 39)

// TODO: confirm the touch threshold with the instructor.
// Lecture Note 3 gives no threshold value. Slide 40 only shows sample output:
// roughly 38–46 when the wire is NOT touched, and roughly 6–9 when it IS touched.
// The value below sits between those two observed bands but is NOT stated in
// the course material. Verify it against your own board before relying on it.
const int touchThreshold = 20;  // UNCONFIRMED - not from the supplied material

void setup() {
  Serial.begin(115200);
  Serial.println();

  // Task 1: connect to WiFi and show the IP address (as in Example 6, slide 48)
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.print("Connected, IP address: ");
  Serial.println(WiFi.localIP());
}

void loop() {
  // Task 2: read the touch pin; a LOW value means the wire is being touched
  // (slide 38: touching returns a low value, not touching returns a high value).
  int touchValue = touchRead(touchPin);
  Serial.println(touchValue);

  if (touchValue < touchThreshold) {
    Serial.println("Touched - restarting");
    ESP.restart();  // software reset (slide 43)
  }

  delay(1000);  // same 1 second sampling period as Example 4 (slide 40)
}
```

4. Lecture Note 4: multitasking and deep sleep

The shared breadboard for Examples 1–5: **SW → 34** (press = LOW), **LED1 → 16**, **LED2 → 17**. Task 1: LED1 on 2 s / off 2 s. Task 2: LED2 follows the switch. (slide 4)

LN4 command reference

Do this	Command	Notes
Elapsed time	`X = millis();`	milliseconds since start; does not block
Interrupt	`attachInterrupt(digitalPinToInterrupt(GPIO), function, mode);`	in `setup()`; pin must be `INPUT`
Interrupt modes	`LOW` `HIGH` `CHANGE` `FALLING` `RISING`	FALLING = HIGH→LOW = the moment a pull-up switch is pressed
Task function	`void Name(void * parameter) { for (;;) { … } vTaskDelete(NULL); }`	`for (;;)` makes it run forever
Create a task	`xTaskCreate(Name, "Name", 10000, NULL, 1, NULL);`	function, name, stack bytes, parameter, priority 0–24, handle
Task on a core	`xTaskCreatePinnedToCore(Name, "Name", 10000, NULL, 1, NULL, 0);`	last argument: core **0** or **1**
Which core	`xPortGetCoreID()`	
Survive sleep	`RTC_DATA_ATTR int bootCount = 0;`	global; kept in RTC memory
Sleep	`esp_deep_sleep_start();`	nothing after it runs; waking is a **reset**
Wake by timer	`esp_sleep_enable_timer_wakeup(X*1000000);`	X in seconds
Wake by touch	`touchAttachInterrupt(Tpin, callback, Threshold);` then `esp_sleep_enable_touchpad_wakeup();`	both, in that order
Wake by one pin	`esp_sleep_enable_ext0_wakeup(GPIO_NUM_32, LOW);`	LOW or HIGH; no `pinMode` needed
Wake by several pins	`esp_sleep_enable_ext1_wakeup(BUTTON_PIN_MASK, LOGIC_LEVEL);`	`ESP_EXT1_WAKEUP_ANY_HIGH` or `ESP_EXT1_WAKEUP_ALL_LOW`

Rule for every wake-up source: call the `esp_sleep_enable_…` line **before** `esp_deep_sleep_start()`. Sources can be combined; any one of them wakes the board. (slide 48)

4.1 Example 1: millis

LED1 blinks every 2 s while LED2 follows the switch, with no `delay()`. (slide 10)

Sketch: `sketches/LN4_Ex1_millis/LN4_Ex1_millis.ino`

```
// set pin numbers
const int buttonPin = 34;  // the push button pin
const int led1Pin = 16;    // the LED1 pin
const int led2Pin = 17;    // the LED2 pin

// variable for storing the pushbutton status
int buttonState = LOW;
int led1State = LOW;
unsigned long previousMillis = 0;
const long interval = 2000; // 2 second

void setup() {
  pinMode(buttonPin, INPUT);
  pinMode(led1Pin, OUTPUT);
  pinMode(led2Pin, OUTPUT);
}

void loop() {
  // To turn on/off the LED1
  unsigned long currentMillis = millis();
  if (currentMillis - previousMillis >= interval) {
    // save the last time you blinked the LED
    previousMillis = currentMillis;
    if (led1State == LOW) {
      led1State = HIGH;
    } else {
      led1State = LOW;
    }
    digitalWrite(led1Pin, led1State);
  }

  // To turn on/off the LED2
  buttonState = digitalRead(buttonPin);
  if (buttonState == LOW) { // Press the switch
    digitalWrite(led2Pin, HIGH); // LED2 on
  } else {
    digitalWrite(led2Pin, LOW); // LED2 off
  }
}
```

The pattern, reusable for anything periodic:

```
unsigned long currentMillis = millis();
if (currentMillis - previousMillis >= interval) {
  previousMillis = currentMillis;
  // do the periodic thing
}
```

4.2 Example 2: pin-change interrupt

LED1 blinks; each press **toggles** LED2. (slides 16–17)

Sketch: `sketches/LN4_Ex2_Interrupts/LN4_Ex2_Interrupts.ino`

```
// set pin numbers
const int buttonPin = 34;  // the push button pin
const int led1Pin = 16;    // the LED1 pin
const int led2Pin = 17;    // the LED2 pin

// variable for storing the pushbutton status
int led1State = LOW;
int led2State = LOW;
unsigned long previousMillis = 0;
const long interval = 2000; // 2 second

void toggle() {
  led2State = !led2State;
  digitalWrite(led2Pin, led2State);
}

void setup() {
  pinMode(buttonPin, INPUT);
  pinMode(led1Pin, OUTPUT);
  pinMode(led2Pin, OUTPUT);
  attachInterrupt(digitalPinToInterrupt(buttonPin), toggle, FALLING);
}

void loop() {
  // To turn on/off the LED1
  unsigned long currentMillis = millis();
  if (currentMillis - previousMillis >= interval) {
    // save the last time you blinked the LED
    previousMillis = currentMillis;
    if (led1State == LOW) {
      led1State = HIGH;
    } else {
      led1State = LOW;
    }
    digitalWrite(led1Pin, led1State);
  }
}
```

Step 1: write the function (`toggle`). Step 2: `attachInterrupt(…)` in `setup()` . There is no debounce, so a bouncy press can toggle twice — that is the instructor's behaviour.

4.3 Example 3: tasks

Same two jobs, each as its own task. `loop()` stays empty. (slides 25–26)

Sketch: `sketches/LN4_Ex3_Multitasking/LN4_Ex3_Multitasking.ino`

```
// set pin numbers
const int buttonPin = 34;  // the push button pin
const int led1Pin = 16;    // the LED1 pin
const int led2Pin = 17;    // the LED2 pin

// variable for storing the pushbutton status
int buttonState = LOW;
int led1State = LOW;
int led2State = LOW;
unsigned long previousMillis = 0;
const long interval = 2000; // 2 second

void TimerLED(void * parameter){
  for (;;) {
    unsigned long currentMillis = millis();
    if (currentMillis - previousMillis >= interval) {
      // save the last time you blinked the LED
      previousMillis = currentMillis;
      if (led1State == LOW) {
        led1State = HIGH;
      } else {
        led1State = LOW;
      }
    }
    digitalWrite(led1Pin, led1State);
  }
  vTaskDelete(NULL);
}

void SwitchLED(void * parameter) {
  for (;;) {
    buttonState = digitalRead(buttonPin);
    if (buttonState == LOW) { // Press the switch
      digitalWrite(led2Pin, HIGH); // LED2 on
    } else {
      digitalWrite(led2Pin, LOW); // LED2 off
    }
  }
  vTaskDelete(NULL);
}

void setup() {
  pinMode(buttonPin, INPUT);
  pinMode(led1Pin, OUTPUT);
  pinMode(led2Pin, OUTPUT);
  xTaskCreate(TimerLED, "TimerLED", 1024, NULL, 1, NULL);
  xTaskCreate(SwitchLED, "SwitchLED", 1024, NULL, 1, NULL);
}

void loop() {
}
```

4.4 Example 4: show the running core

Example 3 plus a print of the core number in each task. (slides 30–31; lines 1–12 are printed only on slide 25)

Sketch: `sketches/LN4_Ex4_ShowCore/LN4_Ex4_ShowCore.ino`

```
// set pin numbers
const int buttonPin = 34;  // the push button pin
const int led1Pin = 16;    // the LED1 pin
const int led2Pin = 17;    // the LED2 pin

// variable for storing the pushbutton status
int buttonState = LOW;
int led1State = LOW;
int led2State = LOW;
unsigned long previousMillis = 0;
const long interval = 2000; // 2 second

void TimerLED(void * parameter){
  for (;;) {
    Serial.print("TimerLED() running on core ");
    Serial.println(xPortGetCoreID());
    delay(100);
    unsigned long currentMillis = millis();
    if (currentMillis - previousMillis >= interval) {
      // save the last time you blinked the LED
      previousMillis = currentMillis;
      if (led1State == LOW) {
        led1State = HIGH;
      } else {
        led1State = LOW;
      }
    }
    digitalWrite(led1Pin, led1State);
  }
  vTaskDelete(NULL);
}

void SwitchLED(void * parameter) {
  for (;;) {
    Serial.print("SwitchLED() running on core ");
    Serial.println(xPortGetCoreID());
    delay(100);
    buttonState = digitalRead(buttonPin);
    if (buttonState == LOW) { // Press the switch
      digitalWrite(led2Pin, HIGH); // LED2 on
    } else {
      digitalWrite(led2Pin, LOW); // LED2 off
    }
  }
  vTaskDelete(NULL);
}

void setup() {
  Serial.begin(115200);
  pinMode(buttonPin, INPUT);
  pinMode(led1Pin, OUTPUT);
  pinMode(led2Pin, OUTPUT);
  xTaskCreate(TimerLED, "TimerLED", 1024, NULL, 1, NULL);
  xTaskCreate(SwitchLED, "SwitchLED", 1024, NULL, 1, NULL);
}
```

```
void loop() {
}
```

4.5 Example 5: assign a core

TimerLED on **core 0**, SwitchLED on **core 1**. (slides 35–36)

Sketch: `sketches/LN4_Ex5_AssignCore/LN4_Ex5_AssignCore.ino`

```
// set pin numbers
const int buttonPin = 34;  // the push button pin
const int led1Pin = 16;    // the LED1 pin
const int led2Pin = 17;    // the LED2 pin

// variable for storing the pushbutton status
int buttonState = LOW;
int led1State = LOW;
int led2State = LOW;
unsigned long previousMillis = 0;
const long interval = 2000; // 2 second

void TimerLED(void * parameter){
  for (;;) {
    Serial.print("TimerLED() running on core ");
    Serial.println(xPortGetCoreID());
    delay(100);
    unsigned long currentMillis = millis();
    if (currentMillis - previousMillis >= interval) {
      // save the last time you blinked the LED
      previousMillis = currentMillis;
      if (led1State == LOW) {
        led1State = HIGH;
      } else {
        led1State = LOW;
      }
    }
    digitalWrite(led1Pin, led1State);
  }
  vTaskDelete(NULL);
}

void SwitchLED(void * parameter) {
  for (;;) {
    Serial.print("SwitchLED() running on core ");
    Serial.println(xPortGetCoreID());
    delay(100);
    buttonState = digitalRead(buttonPin);
    if (buttonState == LOW) { // Press the switch
      digitalWrite(led2Pin, HIGH); // LED2 on
    } else {
      digitalWrite(led2Pin, LOW); // LED2 off
    }
  }
  vTaskDelete(NULL);
}

void setup() {
  Serial.begin(115200);
  pinMode(buttonPin, INPUT);
  pinMode(led1Pin, OUTPUT);
  pinMode(led2Pin, OUTPUT);
  xTaskCreatePinnedToCore(TimerLED, "TimerLED", 1024, NULL, 1, NULL, 0);
  xTaskCreatePinnedToCore(SwitchLED, "SwitchLED", 1024, NULL, 1, NULL, 1);
}
```

```
void loop() {
}
```

Only the two `xTaskCreate…` lines differ from Example 4. If the board resets with a stack-overflow message, raise `1024` to `10000` — the value the lecture's own syntax slide uses (slide 23).

4.6 Example 6: deep sleep

Sleeps forever after 10 s. No circuit. Only RESET wakes it. (slide 46)

Sketch: `sketches/LN4_Ex6_DeepSleep/LN4_Ex6_DeepSleep.ino`

```
void setup(){
  Serial.begin(115200);
  delay(1000); //Take some time to open up the Serial Monitor
  Serial.println("ESP32 will enter the deep sleep in 10 s.");
  delay(10000);
  //Go to sleep now
  Serial.println("... Now, in the deep sleep mode.");
  esp_deep_sleep_start();
  Serial.println("This sentence will never be printed.");
}

void loop(){
  //ESP32 will never come here.
}
```

4.7 Example 7: timer wake-up

Awake 10 s, asleep 5 s, repeating. `bootCount` survives each sleep. No circuit. (slide 50)

Sketch: `sketches/LN4_Ex7_TimerWakeUp/LN4_Ex7_TimerWakeUp.ino`

```
#define TIME_TO_SLEEP 5
RTC_DATA_ATTR int bootCount = 0;

void setup(){
  // Declare the timer wake up
  esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP*1000000);

  // Tasks to do before going to deep sleep
  Serial.begin(115200);
  delay(1000); //Take some time to open up the Serial Monitor
  ++bootCount;
  Serial.println("Boot number: " + String(bootCount));
  Serial.println("ESP32 will enter the deep sleep in 10 s.");
  delay(10000);

  //Go to sleep now
  Serial.println("... Now, in the deep sleep mode.");
  esp_deep_sleep_start();
  Serial.println("This sentence will never be printed.");
}

void loop(){
  //ESP32 will never come here.
}
```

4.8 Example 8: touch wake-up

Sleeps, wakes when the wire on GPIO 32 is touched. (slide 54)

Sketch: `sketches/LN4_Ex8_TouchWakeUp/LN4_Ex8_TouchWakeUp.ino`

```
#define Threshold 40 // higher value, higer sensitivity
RTC_DATA_ATTR int bootCount = 0;

void callback(){
  //placeholder callback function
}
void setup(){
  // Declare the touch wake up - use GPIO32 (TOUCH9)
  touchAttachInterrupt(32, callback, Threshold);
  esp_sleep_enable_touchpad_wakeup();

  // Tasks to do before going to deep sleep
  Serial.begin(115200);
  delay(1000); //Take some time to open up the Serial Monitor
  ++bootCount;
  Serial.println("Boot number: " + String(bootCount));
  Serial.println("ESP32 will enter the deep sleep in 10 s.");
  delay(10000);

  //Go to sleep now
  Serial.println("... Now, in the deep sleep mode.");
  esp_deep_sleep_start();
  Serial.println("This sentence will never be printed.");
}
void loop(){
  //ESP32 will never come here.
}
```

The interrupt fires when the touch value drops **below** `Threshold` (slide 52). The callback can be empty but must exist.

4.9 Example 9: ext0 wake-up

Sleeps, wakes when the switch on GPIO 32 is pressed (LOW). Circuit: SW → 32 with 10 kΩ pull-up. (slide 58)

Sketch: `sketches/LN4_Ex9_ext0WakeUp/LN4_Ex9_ext0WakeUp.ino`

```
RTC_DATA_ATTR int bootCount = 0;

void setup(){
  // Declare the ext0 wake up - use GPIO32
  esp_sleep_enable_ext0_wakeup(GPIO_NUM_32,LOW);

  // Tasks to do before going to deep sleep
  Serial.begin(115200);
  delay(1000); //Take some time to open up the Serial Monitor
  ++bootCount;
  Serial.println("Boot number: " + String(bootCount));
  Serial.println("ESP32 will enter the deep sleep in 10 s.");
  delay(10000);

  //Go to sleep now
  Serial.println("... Now, in the deep sleep mode.");
  esp_deep_sleep_start();
  Serial.println("This sentence will never be printed.");
}

void loop(){
  //ESP32 will never come here.
}
```

4.10 Example 10: ext1 wake-up

Sleeps, wakes when **either** switch is pressed. Circuit: SW1 → 2, SW2 → 32, wired so pressing gives HIGH (see the caution in 2.4). (slide 65)

Sketch: `sketches/LN4_Ex10_ext1WakeUp/LN4_Ex10_ext1WakeUp.ino`

```
RTC_DATA_ATTR int bootCount = 0;

void setup(){
  // Declare the ext1 wake up - use GPIO2 and GPIO32
  esp_sleep_enable_ext1_wakeup(0x0100000004, ESP_EXT1_WAKEUP_ANY_HIGH);

  // Tasks to do before going to deep sleep
  Serial.begin(115200);
  delay(1000); //Take some time to open up the Serial Monitor
  ++bootCount;
  Serial.println("Boot number: " + String(bootCount));
  Serial.println("ESP32 will enter the deep sleep in 10 s.");
  delay(10000);

  //Go to sleep now
  Serial.println("... Now, in the deep sleep mode.");
  esp_deep_sleep_start();
  Serial.println("This sentence will never be printed.");
}

void loop(){
  //ESP32 will never come here.
}
```

Building `BUTTON_PIN_MASK`: one bit per GPIO, bit 0 = GPIO 0 … bit 39 = GPIO 39. Add the values of the pins you use. (slide 62)

GPIO	Hex value		GPIO	Hex value
0	`0x1`		26	`0x4000000`
2	`0x4`		27	`0x8000000`
4	`0x10`		32	`0x100000000`
12	`0x1000`		33	`0x200000000`
13	`0x2000`		34	`0x400000000`
14	`0x4000`		35	`0x800000000`
15	`0x8000`		36	`0x1000000000`
25	`0x2000000`		39	`0x8000000000`

Checks against the lecture: GPIO 32 + 33 = `0x0300000000` (slide 62); GPIO 2 + 32 = `0x0100000004` (slide 64).

4.11 Homework 2 Problem 3

Example 5 plus: pressing **SW1 (GPIO 26)** sends the board to deep sleep through a pin-change interrupt, and a timer wakes it **15 s** later. *(Composed for the homework from Examples 2, 5 and 7.)*

Sketch:
`sketches/HW2_P3_Tasks_SleepButton_TimerWake/HW2_P3_Tasks_SleepButton_TimerWake.ino`

```
#define TIME_TO_SLEEP 15   // wake up 15 seconds after entering deep sleep
RTC_DATA_ATTR int bootCount = 0;

// set pin numbers
const int buttonPin = 34;      // the push button pin (SW)
const int sleepButtonPin = 26; // the deep-sleep button pin (SW1)
const int led1Pin = 16;        // the LED1 pin
const int led2Pin = 17;        // the LED2 pin

// variable for storing the pushbutton status
int buttonState = LOW;
int led1State = LOW;
int led2State = LOW;
unsigned long previousMillis = 0;
const long interval = 2000; // 2 second

// Set by the SW1 interrupt; acted on in loop(). Keeping the handler this small
// follows the taught approach of a minimal interrupt function (Example 2).
volatile bool sleepRequested = false;

void requestSleep() {
  sleepRequested = true;
}

void TimerLED(void * parameter){
  for (;;) {
    Serial.print("TimerLED() running on core ");
    Serial.println(xPortGetCoreID());
    delay(100);
    unsigned long currentMillis = millis();
    if (currentMillis - previousMillis >= interval) {
      // save the last time you blinked the LED
      previousMillis = currentMillis;
      if (led1State == LOW) {
        led1State = HIGH;
      } else {
        led1State = LOW;
      }
    }
    digitalWrite(led1Pin, led1State);
  }
  vTaskDelete(NULL);
}

void SwitchLED(void * parameter) {
  for (;;) {
    Serial.print("SwitchLED() running on core ");
    Serial.println(xPortGetCoreID());
    delay(100);
    buttonState = digitalRead(buttonPin);
    if (buttonState == LOW) { // Press the switch
      digitalWrite(led2Pin, HIGH); // LED2 on
    } else {
      digitalWrite(led2Pin, LOW); // LED2 off
    }
  }
```

```
  vTaskDelete(NULL);
}

void setup() {
  Serial.begin(115200);
  delay(1000); //Take some time to open up the Serial Monitor
  ++bootCount;
  Serial.println("Boot number: " + String(bootCount));

  pinMode(buttonPin, INPUT);
  pinMode(sleepButtonPin, INPUT);
  pinMode(led1Pin, OUTPUT);
  pinMode(led2Pin, OUTPUT);

  // (New) pressing SW1 requests deep sleep, via a pin-change interrupt
  attachInterrupt(digitalPinToInterrupt(sleepButtonPin), requestSleep, FALLING);

  // (New) declare the timer wake up, before going to deep sleep
  esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP*1000000);

  // Declare the tasks and assign the cores.
  xTaskCreatePinnedToCore(TimerLED, "TimerLED", 1024, NULL, 1, NULL, 0);
  xTaskCreatePinnedToCore(SwitchLED, "SwitchLED", 1024, NULL, 1, NULL, 1);
}

void loop() {
  if (sleepRequested) {
    Serial.println("SW1 pressed ... Now, in the deep sleep mode.");
    delay(100); // let the serial output leave the buffer before the CPU powers down
    esp_deep_sleep_start();
  }
}
```

The handler only sets a flag; `loop()` does the sleeping. To change the wake source, swap the one `esp_sleep_enable_…` line.

5. Lecture Note 5: web servers and clients

LN5 essentials

- The ESP32 **is** the web server. Browser and ESP32 must be on the **same WiFi network**. Type the IP printed on the serial monitor into the browser. (slides 40, 43)
- The page is not a file. The sketch prints it with `client.println()` on every request. (slide 65)
- A button is a link: `<a href="/26/on">`. Pressing it sends `GET /26/on`, and the sketch finds that text with `header.indexOf("GET /26/on") >= 0`. `indexOf` returns –1 when the text is absent. (slides 63–64)
- **Nothing updates by itself.** A sensor value changes only when the page is reloaded. (slide 83)

Do this	Command
Server on port 80	`WiFiServer server(80);` then `server.begin();`
Wait for a browser	`WiFiClient client = server.available();`
Read the request	`char c = client.read(); header += c;`
Route	`if (header.indexOf("GET /26/on") >= 0) { … }`
Send the page	`client.println("…");`
Finish	`header = ""; client.stop();`
HTTP client	`#include <HTTPClient.h>`, `HTTPClient http;`
Send a GET	`http.begin(client, url); int code = http.GET(); http.end();`

HTML used: `<h1>` heading · `<p>` paragraph · `<a href="…">` link · `<button class="button">` · `<table>`, `<tr>` row, `<th>` header cell, `<td>` cell · CSS in `<style>` · `.button` green `#4CAF50`, `.button2` grey `#555555`.

5.1 Section 3: web server controlling LEDs

Two buttons, two LEDs. Circuit: LED1 → 26, LED2 → 27. (slides 52–59)

Sketch:
`sketches/LN5_S3_WebServer_ControlOutputs/LN5_S3_WebServer_ControlOutputs.ino`

```
// Load Wi-Fi library
#include <WiFi.h>

// Replace with your network credentials
const char* ssid = "Your SSID";
const char* password = "Your Password";

// Set web server port number to 80
WiFiServer server(80);

// Variable to store the HTTP request
String header;

// Auxiliar variables to store the current output state
String output26State = "off";
String output27State = "off";

// Assign output variables to GPIO pins
const int output26 = 26;
const int output27 = 27;

// Current time
unsigned long currentTime = millis();
// Previous time
unsigned long previousTime = 0;
// Define timeout time in milliseconds (example: 2000ms = 2s)
const long timeoutTime = 2000;

void setup() {
  Serial.begin(115200);
  // Initialize the output variables as outputs
  pinMode(output26, OUTPUT);
  pinMode(output27, OUTPUT);
  // Set outputs to LOW
  digitalWrite(output26, LOW);
  digitalWrite(output27, LOW);

  // Connect to Wi-Fi network with SSID and password
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  // Print local IP address and start web server
  Serial.println("");
  Serial.println("WiFi connected.");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
  server.begin();
}

void loop(){
  WiFiClient client = server.available();   // Listen for incoming clients
```

```
  if (client) {                             // If a new client connects,
    currentTime = millis();
    previousTime = currentTime;
    Serial.println("New Client.");          // print a message out in the serial port
    String currentLine = "";                // make a String to hold incoming data from the client
    while (client.connected() && currentTime - previousTime <= timeoutTime) {  // loop while the client's connected
      currentTime = millis();
      if (client.available()) {             // if there's bytes to read from the client,
        char c = client.read();             // read a byte, then
        Serial.write(c);                    // print it out the serial monitor
        header += c;
        if (c == '\n') {                    // if the byte is a newline character
          // if the current line is blank, you got two newline characters in a row.
          // that's the end of the client HTTP request, so send a response:
          if (currentLine.length() == 0) {
            // HTTP headers always start with a response code (e.g. HTTP/1.1 200 OK)
            // and a content-type so the client knows what's coming, then a blank line:
            client.println("HTTP/1.1 200 OK");
            client.println("Content-type:text/html");
            client.println("Connection: close");
            client.println();

            // turns the GPIOs on and off
            if (header.indexOf("GET /26/on") >= 0) {
              Serial.println("GPIO 26 on");
              output26State = "on";
              digitalWrite(output26, HIGH);
            } else if (header.indexOf("GET /26/off") >= 0) {
              Serial.println("GPIO 26 off");
              output26State = "off";
              digitalWrite(output26, LOW);
            } else if (header.indexOf("GET /27/on") >= 0) {
              Serial.println("GPIO 27 on");
              output27State = "on";
              digitalWrite(output27, HIGH);
            } else if (header.indexOf("GET /27/off") >= 0) {
              Serial.println("GPIO 27 off");
              output27State = "off";
              digitalWrite(output27, LOW);
            }

            // Display the HTML web page
            client.println("<!DOCTYPE html><html>");
            client.println("<head><meta name=\"viewport\" content=\"width=device-width, initial-scale=1\">");
            client.println("<link rel=\"icon\" href=\"data:,\">");
            // CSS to style the on/off buttons
            // Feel free to change the background-color and font-size attributes to fit your preferences
            client.println("<style>html { font-family: Helvetica; display: inline-block; margin: 0px auto; text-align:
center;}");
            client.println(".button { background-color: #4CAF50; border: none; color: white; padding: 16px 40px;");
            client.println("text-decoration: none; font-size: 30px; margin: 2px; cursor: pointer;}");
            client.println(".button2 {background-color: #555555;}</style></head>");

            // Web Page Heading
            client.println("<body><h1>ESP32 Web Server</h1>");
```

```
            // Display current state, and ON/OFF buttons for GPIO 26
            client.println("<p>GPIO 26 - State " + output26State + "</p>");
            // If the output26State is off, it displays the ON button
            if (output26State=="off") {
              client.println("<p><a href=\"/26/on\"><button class=\"button\">ON</button></a></p>");
            } else {
              client.println("<p><a href=\"/26/off\"><button class=\"button button2\">OFF</button></a></p>");
            }

            // Display current state, and ON/OFF buttons for GPIO 27
            client.println("<p>GPIO 27 - State " + output27State + "</p>");
            // If the output27State is off, it displays the ON button
            if (output27State=="off") {
              client.println("<p><a href=\"/27/on\"><button class=\"button\">ON</button></a></p>");
            } else {
              client.println("<p><a href=\"/27/off\"><button class=\"button button2\">OFF</button></a></p>");
            }
            client.println("</body></html>");

            // The HTTP response ends with another blank line
            client.println();
            // Break out of the while loop
            break;
          } else { // if you got a newline, then clear currentLine
            currentLine = "";
          }
        } else if (c != '\r') {  // if you got anything else but a carriage return character,
          currentLine += c;      // add it to the end of the currentLine
        }
      }
    }
    // Clear the header variable
    header = "";
    // Close the connection
    client.stop();
    Serial.println("Client disconnected.");
    Serial.println("");
  }
}
```

5.2 Section 4: web server showing a sensor

A table with the potentiometer value; press refresh to update. Circuit: pot → A0. (slides 76–82)

Sketch:
`sketches/LN5_S4_WebServer_MonitorSensor/LN5_S4_WebServer_MonitorSensor.ino`

```
// Load Wi-Fi library
#include <WiFi.h>

// Replace with your network credentials
const char* ssid = "Your SSID";
const char* password = "Your Password";

// Set web server port number to 80
WiFiServer server(80);

// Variable to store the HTTP request
String header;

// Current time
unsigned long currentTime = millis();
// Previous time
unsigned long previousTime = 0;
// Define timeout time in milliseconds (example: 2000ms = 2s)
const long timeoutTime = 2000;

void setup() {
  Serial.begin(115200);
  // Connect to Wi-Fi network with SSID and password
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  // Print local IP address and start web server
  Serial.println("");
  Serial.println("WiFi connected.");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
  server.begin();
}

void loop(){
  WiFiClient client = server.available();   // Listen for incoming clients

  if (client) {                             // If a new client connects,
    currentTime = millis();
    previousTime = currentTime;
    Serial.println("New Client.");          // print a message out in the serial port
    String currentLine = "";                // make a String to hold incoming data from the client
    while (client.connected() && currentTime - previousTime <= timeoutTime) {  // loop while the client's connected
      currentTime = millis();
      if (client.available()) {             // if there's bytes to read from the client,
        char c = client.read();             // read a byte, then
        Serial.write(c);                    // print it out the serial monitor
        header += c;
        if (c == '\n') {                    // if the byte is a newline character
          // if the current line is blank, you got two newline characters in a row.
          // that's the end of the client HTTP request, so send a response:
```

```
          if (currentLine.length() == 0) {
            // HTTP headers always start with a response code (e.g. HTTP/1.1 200 OK)
            // and a content-type so the client knows what's coming, then a blank line:
            client.println("HTTP/1.1 200 OK");
            client.println("Content-type:text/html");
            client.println("Connection: close");
            client.println();

            // Display the HTML web page
            client.println("<!DOCTYPE html><html>");
            client.println("<head><meta name=\"viewport\" content=\"width=device-width, initial-scale=1\">");
            client.println("<link rel=\"icon\" href=\"data:,\">");
            // CSS to style the on/off buttons
            // Feel free to change the background-color and font-size attributes to fit your preferences
            client.println("<style>html { font-family: Helvetica; display: inline-block; margin: 0px auto; text-align:
center;}");
            client.println("table, th, td { border: 1px solid black; margin-left:auto; margin-right:auto;}");
            client.println("</style></head>");

            // Web Page Heading
            client.println("<body><h1>ESP32 Web Server</h1>");

            // Create a table
            client.println("<table>");
            client.println("<tr><th>Sensor</th><th>Value</th></tr>"); // First row
            client.print("<tr><td>Potentiometer</td><td>"); // Second row
            client.print(analogRead(A0)); // Read the potentiometer value
            client.println("</td></tr>"); // Second row
            client.println("</table>");
            client.println("</body></html>");

            // The HTTP response ends with another blank line
            client.println();
            // Break out of the while loop
            break;
          } else { // if you got a newline, then clear currentLine
            currentLine = "";
          }
        } else if (c != '\r') {  // if you got anything else but a carriage return character,
          currentLine += c;      // add it to the end of the currentLine
        }
      }
    }
    // Clear the header variable
    header = "";
    // Close the connection
    client.stop();
    Serial.println("Client disconnected.");
    Serial.println("");
  }
}
```

5.3 Homework 3 Problem 3: three LEDs

Section 3 revised: LED1 → 16, LED2 → 17, LED3 → 32, three buttons. *(Written for the homework.)*

Sketch:
`sketches/HW3_P3_WebServer_ThreeLEDs/HW3_P3_WebServer_ThreeLEDs.ino`

```
// Load Wi-Fi library
#include <WiFi.h>

// Replace with your network credentials
const char* ssid = "Your SSID";
const char* password = "Your Password";

// Set web server port number to 80
WiFiServer server(80);

// Variable to store the HTTP request
String header;

// Auxiliar variables to store the current output state
String output16State = "off";
String output17State = "off";
String output32State = "off";

// Assign output variables to GPIO pins
const int output16 = 16;
const int output17 = 17;
const int output32 = 32;

// Current time
unsigned long currentTime = millis();
// Previous time
unsigned long previousTime = 0;
// Define timeout time in milliseconds (example: 2000ms = 2s)
const long timeoutTime = 2000;

void setup() {
  Serial.begin(115200);
  // Initialize the output variables as outputs
  pinMode(output16, OUTPUT);
  pinMode(output17, OUTPUT);
  pinMode(output32, OUTPUT);
  // Set outputs to LOW
  digitalWrite(output16, LOW);
  digitalWrite(output17, LOW);
  digitalWrite(output32, LOW);

  // Connect to Wi-Fi network with SSID and password
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  // Print local IP address and start web server
  Serial.println("");
  Serial.println("WiFi connected.");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
  server.begin();
```

```
}

void loop(){
  WiFiClient client = server.available();   // Listen for incoming clients

  if (client) {                             // If a new client connects,
    currentTime = millis();
    previousTime = currentTime;
    Serial.println("New Client.");          // print a message out in the serial port
    String currentLine = "";                // make a String to hold incoming data from the client
    while (client.connected() && currentTime - previousTime <= timeoutTime) {  // loop while the client's connected
      currentTime = millis();
      if (client.available()) {             // if there's bytes to read from the client,
        char c = client.read();             // read a byte, then
        Serial.write(c);                    // print it out the serial monitor
        header += c;
        if (c == '\n') {                    // if the byte is a newline character
          // if the current line is blank, you got two newline characters in a row.
          // that's the end of the client HTTP request, so send a response:
          if (currentLine.length() == 0) {
            // HTTP headers always start with a response code (e.g. HTTP/1.1 200 OK)
            // and a content-type so the client knows what's coming, then a blank line:
            client.println("HTTP/1.1 200 OK");
            client.println("Content-type:text/html");
            client.println("Connection: close");
            client.println();

            // turns the GPIOs on and off
            if (header.indexOf("GET /16/on") >= 0) {
              Serial.println("GPIO 16 on");
              output16State = "on";
              digitalWrite(output16, HIGH);
            } else if (header.indexOf("GET /16/off") >= 0) {
              Serial.println("GPIO 16 off");
              output16State = "off";
              digitalWrite(output16, LOW);
            } else if (header.indexOf("GET /17/on") >= 0) {
              Serial.println("GPIO 17 on");
              output17State = "on";
              digitalWrite(output17, HIGH);
            } else if (header.indexOf("GET /17/off") >= 0) {
              Serial.println("GPIO 17 off");
              output17State = "off";
              digitalWrite(output17, LOW);
            } else if (header.indexOf("GET /32/on") >= 0) {
              Serial.println("GPIO 32 on");
              output32State = "on";
              digitalWrite(output32, HIGH);
            } else if (header.indexOf("GET /32/off") >= 0) {
              Serial.println("GPIO 32 off");
              output32State = "off";
              digitalWrite(output32, LOW);
            }

            // Display the HTML web page
            client.println("<!DOCTYPE html><html>");
            client.println("<head><meta name=\"viewport\" content=\"width=device-width, initial-scale=1\">");
```

```
            client.println("<link rel=\"icon\" href=\"data:,\">");
            // CSS to style the on/off buttons
            // Feel free to change the background-color and font-size attributes to fit your preferences
            client.println("<style>html { font-family: Helvetica; display: inline-block; margin: 0px auto; text-align:
center;}");
            client.println(".button { background-color: #4CAF50; border: none; color: white; padding: 16px 40px;");
            client.println("text-decoration: none; font-size: 30px; margin: 2px; cursor: pointer;}");
            client.println(".button2 {background-color: #555555;}</style></head>");

            // Web Page Heading
            client.println("<body><h1>ESP32 Web Server</h1>");

            // Display current state, and ON/OFF buttons for GPIO 16
            client.println("<p>GPIO 16 - State " + output16State + "</p>");
            // If the output16State is off, it displays the ON button
            if (output16State=="off") {
              client.println("<p><a href=\"/16/on\"><button class=\"button\">ON</button></a></p>");
            } else {
              client.println("<p><a href=\"/16/off\"><button class=\"button button2\">OFF</button></a></p>");
            }

            // Display current state, and ON/OFF buttons for GPIO 17
            client.println("<p>GPIO 17 - State " + output17State + "</p>");
            // If the output17State is off, it displays the ON button
            if (output17State=="off") {
              client.println("<p><a href=\"/17/on\"><button class=\"button\">ON</button></a></p>");
            } else {
              client.println("<p><a href=\"/17/off\"><button class=\"button button2\">OFF</button></a></p>");
            }

            // Display current state, and ON/OFF buttons for GPIO 32
            client.println("<p>GPIO 32 - State " + output32State + "</p>");
            // If the output32State is off, it displays the ON button
            if (output32State=="off") {
              client.println("<p><a href=\"/32/on\"><button class=\"button\">ON</button></a></p>");
            } else {
              client.println("<p><a href=\"/32/off\"><button class=\"button button2\">OFF</button></a></p>");
            }
            client.println("</body></html>");

            // The HTTP response ends with another blank line
            client.println();
            // Break out of the while loop
            break;
          } else { // if you got a newline, then clear currentLine
            currentLine = "";
          }
        } else if (c != '\r') {  // if you got anything else but a carriage return character,
          currentLine += c;      // add it to the end of the currentLine
        }
      }
    }
    // Clear the header variable
    header = "";
    // Close the connection
    client.stop();
    Serial.println("Client disconnected.");
```

```
    Serial.println("");
  }
}
```

Adding an LED to the Section 3 sketch takes four edits: a `String outputNNState = "off";` and `const int outputNN = NN;` · `pinMode` and `digitalWrite(…, LOW)` in `setup()` · two more `else if (header.indexOf("GET /NN/on") …)` branches · one more state-and-button block in the page.

5.4 Buttons and a sensor on one page

Sections 3 and 4 merged: LED buttons on 26 and 27 plus the potentiometer table. *(Written for this sheet. It is Section 3's file with the table CSS line and the table block from Section 4 added — nothing else changed.)*

Sketch:
`sketches/MIX_WebServer_LEDs_and_Sensor/MIX_WebServer_LEDs_and_Sensor.ino`

```
// Load Wi-Fi library
#include <WiFi.h>

// Replace with your network credentials
const char* ssid = "Your SSID";
const char* password = "Your Password";

// Set web server port number to 80
WiFiServer server(80);

// Variable to store the HTTP request
String header;

// Auxiliar variables to store the current output state
String output26State = "off";
String output27State = "off";

// Assign output variables to GPIO pins
const int output26 = 26;
const int output27 = 27;

// Current time
unsigned long currentTime = millis();
// Previous time
unsigned long previousTime = 0;
// Define timeout time in milliseconds (example: 2000ms = 2s)
const long timeoutTime = 2000;

void setup() {
  Serial.begin(115200);
  // Initialize the output variables as outputs
  pinMode(output26, OUTPUT);
  pinMode(output27, OUTPUT);
  // Set outputs to LOW
  digitalWrite(output26, LOW);
  digitalWrite(output27, LOW);

  // Connect to Wi-Fi network with SSID and password
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  // Print local IP address and start web server
  Serial.println("");
  Serial.println("WiFi connected.");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
  server.begin();
}

void loop(){
  WiFiClient client = server.available();   // Listen for incoming clients
```

```
  if (client) {                             // If a new client connects,
    currentTime = millis();
    previousTime = currentTime;
    Serial.println("New Client.");          // print a message out in the serial port
    String currentLine = "";                // make a String to hold incoming data from the client
    while (client.connected() && currentTime - previousTime <= timeoutTime) {  // loop while the client's connected
      currentTime = millis();
      if (client.available()) {             // if there's bytes to read from the client,
        char c = client.read();             // read a byte, then
        Serial.write(c);                    // print it out the serial monitor
        header += c;
        if (c == '\n') {                    // if the byte is a newline character
          // if the current line is blank, you got two newline characters in a row.
          // that's the end of the client HTTP request, so send a response:
          if (currentLine.length() == 0) {
            // HTTP headers always start with a response code (e.g. HTTP/1.1 200 OK)
            // and a content-type so the client knows what's coming, then a blank line:
            client.println("HTTP/1.1 200 OK");
            client.println("Content-type:text/html");
            client.println("Connection: close");
            client.println();

            // turns the GPIOs on and off
            if (header.indexOf("GET /26/on") >= 0) {
              Serial.println("GPIO 26 on");
              output26State = "on";
              digitalWrite(output26, HIGH);
            } else if (header.indexOf("GET /26/off") >= 0) {
              Serial.println("GPIO 26 off");
              output26State = "off";
              digitalWrite(output26, LOW);
            } else if (header.indexOf("GET /27/on") >= 0) {
              Serial.println("GPIO 27 on");
              output27State = "on";
              digitalWrite(output27, HIGH);
            } else if (header.indexOf("GET /27/off") >= 0) {
              Serial.println("GPIO 27 off");
              output27State = "off";
              digitalWrite(output27, LOW);
            }

            // Display the HTML web page
            client.println("<!DOCTYPE html><html>");
            client.println("<head><meta name=\"viewport\" content=\"width=device-width, initial-scale=1\">");
            client.println("<link rel=\"icon\" href=\"data:,\">");
            // CSS to style the on/off buttons
            // Feel free to change the background-color and font-size attributes to fit your preferences
            client.println("<style>html { font-family: Helvetica; display: inline-block; margin: 0px auto; text-align:
center;}");
            client.println(".button { background-color: #4CAF50; border: none; color: white; padding: 16px 40px;");
            client.println("text-decoration: none; font-size: 30px; margin: 2px; cursor: pointer;}");
            client.println(".button2 {background-color: #555555;}");
            client.println("table, th, td { border: 1px solid black; margin-left:auto; margin-right:auto;}");
            client.println("</style></head>");

            // Web Page Heading
```

```
            client.println("<body><h1>ESP32 Web Server</h1>");

            // Display current state, and ON/OFF buttons for GPIO 26
            client.println("<p>GPIO 26 - State " + output26State + "</p>");
            // If the output26State is off, it displays the ON button
            if (output26State=="off") {
              client.println("<p><a href=\"/26/on\"><button class=\"button\">ON</button></a></p>");
            } else {
              client.println("<p><a href=\"/26/off\"><button class=\"button button2\">OFF</button></a></p>");
            }

            // Display current state, and ON/OFF buttons for GPIO 27
            client.println("<p>GPIO 27 - State " + output27State + "</p>");
            // If the output27State is off, it displays the ON button
            if (output27State=="off") {
              client.println("<p><a href=\"/27/on\"><button class=\"button\">ON</button></a></p>");
            } else {
              client.println("<p><a href=\"/27/off\"><button class=\"button button2\">OFF</button></a></p>");
            }

            // Create a table
            client.println("<table>");
            client.println("<tr><th>Sensor</th><th>Value</th></tr>"); // First row
            client.print("<tr><td>Potentiometer</td><td>"); // Second row
            client.print(analogRead(A0)); // Read the potentiometer value
            client.println("</td></tr>"); // Second row
            client.println("</table>");
            client.println("</body></html>");

            // The HTTP response ends with another blank line
            client.println();
            // Break out of the while loop
            break;
          } else { // if you got a newline, then clear currentLine
            currentLine = "";
          }
        } else if (c != '\r') {  // if you got anything else but a carriage return character,
          currentLine += c;      // add it to the end of the currentLine
        }
      }
    }
    // Clear the header variable
    header = "";
    // Close the connection
    client.stop();
    Serial.println("Client disconnected.");
    Serial.println("");
  }
}
```

5.5 Section 5.2: ThingSpeak

Sends the potentiometer value and a random number every 10 s. Circuit: pot → A0. (slides 96–101)

Sketch:
`sketches/LN5_S5_WebClient_ThingSpeak/LN5_S5_WebClient_ThingSpeak.ino`

```
#include <WiFi.h>
#include <HTTPClient.h>

// Replace with your network credentials
const char* ssid = "Your SSID";
const char* password = "Your Password";

// REPLACE WITH THINGSPEAK.COM API KEY
String serverName = "http://api.thingspeak.com/update?api_key=YourAPIkey";
// EXAMPLE:
//String serverName = "http://api.thingspeak.com/update?api_key=7HQJM49R8JAPR";

// THE DEFAULT TIMER IS SET TO 10 SECONDS FOR TESTING PURPOSES
// For a final application, check the API call limits per hour/minute to avoid getting blocked/banned
unsigned long lastTime = 0;
// Set timer to 10 seconds (10000)
unsigned long timerDelay = 10000;

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);
  Serial.println("Connecting");
  while(WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.print("Connected to WiFi network with IP Address: ");
  Serial.println(WiFi.localIP());
  Serial.println("Timer set to 10 seconds (timerDelay variable), it will take 10 seconds before publishing the first
reading.");
}

void loop() {
  // Send an HTTP GET request
  if ((millis() - lastTime) > timerDelay) {
    // Check WiFi connection status
    if(WiFi.status()== WL_CONNECTED){
      WiFiClient client;
      HTTPClient http;
      String HttpRequest = serverName + "&field1=" + String(analogRead(A0)) + "&field2=" + String(random(100));

      // Create the HTTP Get request to ThingSpeak
      Serial.print("HTTP Request: ");
      Serial.println(HttpRequest);
      http.begin(client, HttpRequest);
      //Another option --> http.begin(client, HttpRequest.c_str());

      // Send HTTP GET request
      int httpResponseCode = http.GET();
      if (httpResponseCode>0) {
        Serial.print("HTTP Response code: ");
        Serial.println(httpResponseCode);
      }
      else {
        Serial.print("Error code: ");
```

```
        Serial.println(httpResponseCode);
      }
      // Free resources
      http.end();
    }
    else {
      Serial.println("WiFi Disconnected");
    }
    lastTime = millis();
  }
}
```

The request it builds (slide 92):

```
http://api.thingspeak.com/update?api_key=YourAPIkey&field1=250&field2=40
```

Four ways to break line 9 — all four happened on this project:

Mistake	Symptom
Pasting ThingSpeak's own example with `GET` in front	`Error code: -1`
`https://` instead of `http://`	`Error code: -1` — a plain `WiFiClient` cannot do TLS
Leaving `&field1=0` on the end	`field1` appears twice; ThingSpeak stores the first, so 0
Someone else's API key	`HTTP Response code: 400`

Line 9 must be exactly `"http://api.thingspeak.com/update?api_key=` + your Write API Key + `"`. The sketch adds `&field1=…&field2=…` itself.

`HTTP Response code: 200` means delivered. The free tier accepts one write per 15 s *(not in the lecture)*, and the sketch sends every 10 s, so some writes are dropped; the charts still fill in. A third value is one more `+ "&field3=" + String(…)` on line 40, with Field 3 ticked in the channel.

6. Lecture Note 6: MQTT

LN6 essentials

- Three parts: **publisher**, **broker**, **subscriber**. Publisher and subscriber never contact each other and do not know each other's IP. The **topic** is what connects them. (slides 5–8)
- Broker: `broker.hivemq.com`, port `1883`. If HiveMQ misbehaves, the lecture says to use Mosquitto instead. (slide 19) *(`test.mosquitto.org` is Mosquitto's public broker — not in the lecture.)*
- `ClientID`**: different on every board, and different from other students.** `Topic`**: identical on the boards that must talk, and different from other students.** Add your student ID to both. (slide 23)
- The two boards do not need the same WiFi and can be anywhere. (slide 17)
- Topics are levels separated by `/`. Wildcards: `+` one level, `#` all levels below. (slides 10–12)
- QoS 0 at most once · QoS 1 at least once · QoS 2 exactly once. Delivered at the lower of publisher's and subscriber's level. (slide 13)

Do this	Command
Headers	`#include <WiFi.h>` `#include <PubSubClient.h>`
Client objects	`WiFiClient ClientNode;` `PubSubClient ClientESP(ClientNode);`
Set the broker	`ClientESP.setServer(mqtt_server, mqtt_port);`
Connect	`ClientESP.connect(ClientID, NULL, NULL)`
Connected?	`ClientESP.connected()`
Publish	`ClientESP.publish(Topic, Val);` — `Val` is a `char` array
Number → text	`char Val[8]; dtostrf(analogRead(A0), 1, 0, Val);`
Subscribe	`ClientESP.setCallback(callback);` then `ClientESP.subscribe(Topic);`
Keep receiving	`ClientESP.loop();` in `loop()` — **subscribers need this**
Receive	`void callback(char* topic, byte* payload, unsigned int length)`
Text → number	`int ValInt = ValString.toInt();`

Wildcard answers for the lecture's six topics (`Home1/Room1..3/temp|humid`):
`Home1/Room1/+` → Room1 temp and humid · `Home1/+/temp` → the three temps ·
`Home1/Room1/#` → Room1 temp and humid · `Home1/#` → all six.

6.1 Section 3: publisher node

Publishes the potentiometer value every 2 s. Circuit: pot → A0. (slides 24–27)

Sketch: `sketches/LN6_S3_PubNode/LN6_S3_PubNode.ino`

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "Your SSID";
const char* password = "Your Password";
const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;
const char* ClientID = "Client_PubNode_YourStudentID";
const char* Topic = "IoT/PotenValue_YourStudentID";
long lastMsg = 0;

WiFiClient ClientNode;
PubSubClient ClientESP(ClientNode);

void setup() {
  Serial.begin(115200);
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.print("Connected, IP address: ");
  Serial.println(WiFi.localIP());

  ClientESP.setServer(mqtt_server, mqtt_port); //Specify the MQTT server and port.
  ConnectMQTT(); //Connect to the MQTT server.
}

void loop() {
  if (!ClientESP.connected()) {
    ConnectMQTT();
  }

  long now = millis();
  if (now - lastMsg > 2000) {
    lastMsg = now;
    char Val[8];
    dtostrf(analogRead(A0), 1, 0, Val);
    ClientESP.publish(Topic, Val);
    Serial.print("Pub: ");
    Serial.println(Val);
  }
}

void ConnectMQTT() {
  while (!ClientESP.connected()) {
    Serial.print("Attempting MQTT connection...");
    // Attempt to connect to the broker
    if (ClientESP.connect(ClientID, NULL, NULL)) {
          Serial.println("connected");
```

```
    } else {
      Serial.print("failed, rc=");
      Serial.print(ClientESP.state());
      Serial.println(" try again in 5 seconds");
      // Wait 5 seconds before retrying
      delay(5000);
    }
  }
}
```

6.2 Section 3: subscriber node

Lights the LED when the received value is above **500**. Circuit: LED → 26. (slides 28–31)

Sketch: `sketches/LN6_S3_SubNode/LN6_S3_SubNode.ino`

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "Your SSID";
const char* password = "Your Password";
const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;
const char* ClientID = "Client_SubNode_YourStudentID";
const char* Topic = "IoT/PotenValue_YourStudentID";
long lastMsg = 0;

WiFiClient ClientNode;
PubSubClient ClientESP(ClientNode);

void setup() {
  Serial.begin(115200);
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.print("Connected, IP address: ");
  Serial.println(WiFi.localIP());

  ClientESP.setServer(mqtt_server, mqtt_port); //Specify the MQTT server and port.
  ClientESP.setCallback(callback);
  ConnectMQTT(); //Connect to the MQTT server.

  pinMode(26, OUTPUT); //Set the pin mode to the pin GPIO26
  digitalWrite(26, LOW);
}

void loop() {
  if (!ClientESP.connected()) {
    ConnectMQTT();
  }
  ClientESP.loop();
}

void ConnectMQTT() {
  while (!ClientESP.connected()) {
    Serial.print("Attempting MQTT connection...");
    // Attempt to connect to the broker
    if (ClientESP.connect(ClientID, NULL, NULL)) {
          Serial.println("connected");
          ClientESP.setCallback(callback);
          ClientESP.subscribe(Topic);
    } else {
      Serial.print("failed, rc=");
      Serial.print(ClientESP.state());
```

```
      Serial.println(" try again in 5 seconds");
      // Wait 5 seconds before retrying
      delay(5000);
    }
  }
}

void callback(char* topic, byte* payload, unsigned int length) {
  Serial.print("Message arrived [");
  Serial.print(topic);
  Serial.print("] ");
  String ValString;
  for (int i = 0; i < length; i++) {
    Serial.print((char)payload[i]);
    ValString += (char)payload[i];
  }
  Serial.println();

  int ValInt = ValString.toInt();
  Serial.print("Sub : ");
  Serial.println(ValInt);
  Serial.println("============== Wait 2 seconds ==================");
  if (ValInt > 500) {
    digitalWrite(26, HIGH);   // Turn the LED on
  } else {
    digitalWrite(26, LOW);  // Turn the LED off
  }
}
```

Line 8 differs between the two nodes here on purpose: the instructor's two files ship the same `ClientID`, and slide 23 says they must differ.

6.3 Section 4: one board, both roles

Publishes its own value and subscribes to the same topic, so the number goes to the broker and comes back to its own LED. Circuit: pot → A0 and LED → 26. (slides 43–47)

Sketch: `sketches/LN6_S4_PubSub/LN6_S4_PubSub.ino`

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "Your SSID";
const char* password = "Your Password";
const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;
const char* ClientID = "Client_YourStudentID";
const char* Topic = "IoT/PotenValue_YourStudentID";
long lastMsg = 0;

WiFiClient ClientNode;
PubSubClient ClientESP(ClientNode);

void setup() {
  Serial.begin(115200);
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.print("Connected, IP address: ");
  Serial.println(WiFi.localIP());

  ClientESP.setServer(mqtt_server, mqtt_port); //Specify the MQTT server and port.
  ClientESP.setCallback(callback);
  ConnectMQTT(); //Connect to the MQTT server.

  pinMode(26, OUTPUT); //Set the pin mode to the pin GPIO26
  digitalWrite(26, LOW);
}

void loop() {
  if (!ClientESP.connected()) {
    ConnectMQTT();
  }
  ClientESP.loop();

  long now = millis();
  if (now - lastMsg > 2000) {
    lastMsg = now;
    char Val[8];
    dtostrf(analogRead(A0), 1, 0, Val);
    ClientESP.publish(Topic, Val);
    Serial.print("Pub: ");
    Serial.println(Val);
  }
}

void ConnectMQTT() {
```

```
  while (!ClientESP.connected()) {
    Serial.print("Attempting MQTT connection...");
    // Attempt to connect to the broker
    if (ClientESP.connect(ClientID, NULL, NULL)) {
          Serial.println("connected");
          ClientESP.setCallback(callback);
          ClientESP.subscribe(Topic);
    } else {
      Serial.print("failed, rc=");
      Serial.print(ClientESP.state());
      Serial.println(" try again in 5 seconds");
      // Wait 5 seconds before retrying
      delay(5000);
    }
  }
}

void callback(char* topic, byte* payload, unsigned int length) {
  Serial.print("Message arrived [");
  Serial.print(topic);
  Serial.print("] ");
  String ValString;
  for (int i = 0; i < length; i++) {
    Serial.print((char)payload[i]);
    ValString += (char)payload[i];
  }
  Serial.println();

  int ValInt = ValString.toInt();
  Serial.print("Sub : ");
  Serial.println(ValInt);
  Serial.println("============== Wait 2 seconds ==================");
  if (ValInt > 500) {
    digitalWrite(26, HIGH);   // Turn the LED on
  } else {
    digitalWrite(26, LOW);  // Turn the LED off
  }
}
```

6.4 Homework 4 Q2: two boards, crossed over

Each board has a potentiometer and an LED; each knob drives the **other** board's LED. One topic becomes two, swapped between the boards. *(Written for the homework.)*

	publishes to	listens on
Node1	`IoT/PotenValue_Node1_<ID>`	`IoT/PotenValue_Node2_<ID>`
Node2	`IoT/PotenValue_Node2_<ID>`	`IoT/PotenValue_Node1_<ID>`

Sketch: `sketches/HW4_Q2_Node1/HW4_Q2_Node1.ino`

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "Your SSID";
const char* password = "Your Password";
const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;
const char* ClientID = "Client_Node1_YourStudentID";
const char* TopicPub = "IoT/PotenValue_Node1_YourStudentID";
const char* TopicSub = "IoT/PotenValue_Node2_YourStudentID";
long lastMsg = 0;

WiFiClient ClientNode;
PubSubClient ClientESP(ClientNode);

void setup() {
  Serial.begin(115200);
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.print("Connected, IP address: ");
  Serial.println(WiFi.localIP());

  ClientESP.setServer(mqtt_server, mqtt_port); //Specify the MQTT server and port.
  ClientESP.setCallback(callback);
  ConnectMQTT(); //Connect to the MQTT server.

  pinMode(26, OUTPUT); //Set the pin mode to the pin GPIO26
  digitalWrite(26, LOW);
}

void loop() {
  if (!ClientESP.connected()) {
    ConnectMQTT();
  }
  ClientESP.loop();

  long now = millis();
  if (now - lastMsg > 2000) {
    lastMsg = now;
    char Val[8];
    dtostrf(analogRead(A0), 1, 0, Val);
    ClientESP.publish(TopicPub, Val);
    Serial.print("Pub: ");
    Serial.println(Val);
  }
}
```

```
void ConnectMQTT() {
  while (!ClientESP.connected()) {
    Serial.print("Attempting MQTT connection...");
    // Attempt to connect to the broker
    if (ClientESP.connect(ClientID, NULL, NULL)) {
          Serial.println("connected");
          ClientESP.setCallback(callback);
          ClientESP.subscribe(TopicSub);
    } else {
      Serial.print("failed, rc=");
      Serial.print(ClientESP.state());
      Serial.println(" try again in 5 seconds");
      // Wait 5 seconds before retrying
      delay(5000);
    }
  }
}

void callback(char* topic, byte* payload, unsigned int length) {
  Serial.print("Message arrived [");
  Serial.print(topic);
  Serial.print("] ");
  String ValString;
  for (int i = 0; i < length; i++) {
    Serial.print((char)payload[i]);
    ValString += (char)payload[i];
  }
  Serial.println();

  int ValInt = ValString.toInt();
  Serial.print("Sub : ");
  Serial.println(ValInt);
  Serial.println("============== Wait 2 seconds ==================");
  if (ValInt > 500) {
    digitalWrite(26, HIGH);   // Turn the LED on
  } else {
    digitalWrite(26, LOW);  // Turn the LED off
  }
}
```

`HW4_Q2_Node2` is the same file with `Node1` and `Node2` exchanged on lines 8–10.

MQTT failures and what they mean

Serial monitor shows	Cause
`Attempting MQTT connection...connected` repeating forever	two boards share one `ClientID`
`Pub:` lines but the other board never prints `Message arrived`	the topics differ by a character, or the subscriber has no `ClientESP.loop()`
LED reacts to nothing you did	another student picked your topic
`failed, rc=-2`	cannot reach the broker — WiFi has no internet, or the broker is down

Serial monitor shows	Cause
`failed, rc=-4`	connection timed out

`rc` values come from the PubSubClient library *(not in the lecture)*: −4 timeout · −3 connection lost · −2 connect failed · −1 disconnected · 0 connected · 2 client ID rejected · 5 not authorised.

7. Changing an example to fit the question

Exam questions in this course have looked like the homework's last problems: an example with one thing changed or two examples joined. The changes that came up:

The question wants	Change
a different pin	the `const int …Pin = NN;` line. Check 2.3 that the pin can do the job
pressing = OFF instead of ON	swap `HIGH` and `LOW` in the `if`, or test `== HIGH`
a different blink time	`const long interval = 2000;`
a different threshold	`if (ValInt > 500)` (MQTT) · `Threshold` (touch)
a different publish period	`if (now – lastMsg > 2000)`
a different ThingSpeak period	`unsigned long timerDelay = 10000;`
a different sleep time	`#define TIME_TO_SLEEP 5`
stay awake longer before sleeping	the `delay(10000);` before `esp_deep_sleep_start();`
a different wake source	swap the one `esp_sleep_enable_…` line (section 4)
wake by switch **or** timer	call both enable lines before sleeping
toggle instead of follow	use the interrupt version (Example 2) with `FALLING`
count something across sleeps	`RTC_DATA_ATTR int count = 0;` and `++count;`
brightness instead of on/off	replace `digitalWrite` with the three PWM steps
touch instead of a switch	`if (touchRead(32) < Threshold)` in place of `digitalRead(buttonPin) == LOW`
publish something other than the pot	change the first argument of `dtostrf(…, 1, 0, Val);` — e.g. `touchRead(32)` or `digitalRead(34)`
show something other than the pot on the web page	change `client.print(analogRead(A0));`
a second sensor row on the web page	copy the three `client.print…` row lines and change the label and the value
an extra LED on the web page	the four edits listed in 5.3
restart on some event	`ESP.restart();` inside the `if`

Joining two examples: globals from both at the top · both `setup()` bodies into one `setup()` (one `Serial.begin`) · both `loop()` bodies into one `loop()`. If either part uses `delay()`, the other part waits for it — use the `millis()` pattern from 4.1 for the periodic part.

8. When it does not work

Symptom	Fix
Upload sits at `Connecting…`	press the **PROGRAM** button on the board (LN3 slide 20)
No port in Tools → Port	USB cable is charge-only, or the CP210x driver is missing (LN3 slide 16)
Serial monitor shows garbage	set it to **115200**
Garbage only at the instant of reset	normal — the boot message is sent at another speed
`'ledcSetup' was not declared`	ESP32 core 3.x — see 3.3
`PubSubClient.h: No such file`	install the PubSubClient library, restart the IDE (LN6 slide 15)
`Bad CPU type in executable` while compiling (Mac)	the Arduino tools need Rosetta — see the README
Dots forever after `Connecting`	wrong SSID/password, or a 5 GHz-only network. Use a 2.4 GHz phone hotspot
`WiFi Disconnected`, random resets	the phone hotspot went to sleep — keep its screen on
Browser cannot open the IP	phone/laptop is not on the same WiFi as the ESP32
Web page value does not change	press refresh — that is the design
LED never lights	long leg to the GPIO; and GPIO 34/35/36/39 cannot be outputs
Switch always reads the same	the 10 kΩ is missing, or the switch is rotated 90° on the breadboard
Analog reads 0 or 4095 only	wiper is not on `A0`, or an outer leg is unconnected
Analog reads nonsense with WiFi on	you used A10–A19; move to A0
Touch value never changes	the pin is not one of 0, 2, 4, 12–15, 27, 32, 33
Board will not wake from sleep	the `esp_sleep_enable_…` line is after `esp_deep_sleep_start()`, or missing
Serial monitor dead after a wake-up	close and reopen it
Task version resets repeatedly	raise the stack `1024` → `10000`
ThingSpeak `Error code: -1`	line 9 has `https://` or `GET` in it (5.5)
ThingSpeak `400`	wrong Write API Key
MQTT reconnects forever	both boards have the same `ClientID`

9. Before you hand in

- [ ] Each question: circuit works, **video recorded** (circuit, code, demo).
- [ ] Code files saved and uploaded to Google Classroom.
- [ ] Videos uploaded, named as the question paper says.
- [ ] Paper work has your name and ID and goes to the instructor.
- [ ] Your WiFi password and API key are readable in the code on camera. Change them afterwards if that matters to you.